Assignment Report: Memory-Mapped Files Implementation in xv6

[Zheng Qixuan] - [123090884]

November 22, 2025

1 Introduction [2']

This assignment implements memory-mapped files (mmap) in the xv6 operating system. The mmap system call allows programs to map files into their address space, enabling file access through memory operations rather than explicit read-/write calls.

My implementation includes the core mmap/munmap system calls with lazy allocation, proper handling of page faults for mapped regions, and integration with the fork system call to ensure child processes correctly inherit memory mappings. The work demonstrates key OS concepts including virtual memory management, page fault handling, and process resource inheritance.

2 Design [5']

2.1 Data Structures

I extended the process structure with a Virtual Memory Area (VMA) array to track memory mappings:

```
struct vma {
  int valid;               // Whether the slot is valid (0 =
      free, 1 = in use)
  uint64 addr;             // Starting virtual address of
      mapped region (page-aligned)
  uint64 length;           // Length of mapped region (bytes)
  int prot;                // Protection flags (PROT_READ,
      PROT_WRITE, etc.)
  int flags;               // Mapping flags (MAP_SHARED,
      MAP_PRIVATE)
  struct file *f;          // Pointer to mapped file
      structure
  struct inode *ip;        // Direct inode pointer
  uint64 offset;           // Offset within the file (bytes)
};
```

Each process maintains an array of VMAs (defined as VMASIZE), allowing multiple concurrent memory mappings.

2.2 System Call Implementation

mmap Implementation: The mmap system call performs the following steps:

1. Validate arguments (address alignment, file permissions)
2. Find an unused VMA slot in the process's VMA array
3. Allocate virtual address space starting at address 0x7000
4. Store mapping metadata without allocating physical pages (lazy allocation)
5. Increase file reference count using filedup()

munmap Implementation: The munmap system call:

1. Locates the VMA containing the target address
2. For MAP_SHARED mappings with write permission, writes dirty pages back to the file
3. Unmaps pages using uvmunmap() and frees physical memory
4. Releases the file reference and marks VMA slot as invalid

2.3 Page Fault Handling

Lazy allocation is implemented in the usertrap() function. When a page fault occurs:

1. Extract the faulting address from the stval register
2. Search process VMAs to determine if the address falls within a mapped region
3. If valid, allocate a physical page and read file content into it
4. Map the page with appropriate permissions from the VMA
5. If invalid address, kill the process

2.4 Fork Integration

To ensure child processes inherit parent mappings, I modified the fork() function:

1. After copying the page table with uvmcopy(), copy the parent's VMA array
2. For each valid VMA, duplicate the structure and increment file reference count
3. This allows child processes to handle page faults on inherited mappings correctly

2.5 Process Cleanup

In the exit() function, I added VMA cleanup logic:

1. Iterate through all valid VMAs before releasing other resources
2. Unmap virtual addresses using uvmunmap() to free physical pages
3. Close file references with fileclose()
4. Mark VMA slots as invalid

This prevents the "freewalk: leaf" panic by ensuring all mapped pages are properly released before the page table is freed.

3 Environment and Execution [2']

3.1 Environment

- Operating System: xv6 (RISC-V version)
- Build System: GNU Make
- Compiler: RISC-V GCC cross-compiler
- Emulator: QEMU for RISC-V

3.2 Execution

To build and test the implementation:

```
# Build xv6
make qemu

# Run tests
mmaptest
```

3.3 Test Results

The implementation successfully passes all required tests:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
mmap_test starting
test mmap f
test mmap f: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test mmap two files
test mmap two files: OK
test mmap offset
test mmap offset: OK
test mmap half page
test mmap half page: OK
mmap_test: ALL OK
fork_test starting
fork_test OK
mmaptest: all tests succeeded
$ QEMU: Terminated
```

The fork_test specifically validates that child processes correctly inherit and can access parent memory mappings, demonstrating the fork integration works properly.

4 Conclusion [2']

This assignment provided hands-on experience with several fundamental operating system concepts. I learned how virtual memory systems implement lazy allocation to optimize memory usage, how page fault handlers bridge virtual and physical memory, and how process resources must be carefully managed across fork operations.

The most challenging aspect was debugging the munmap function. This requires me to understand the whole logic of how memory allocation and virtrual memory work. At first I did not consider the overlap of address and later I sovled it by searching resources about xv6 mmap design.

Key takeaways include: (1) the power of lazy allocation in reducing memory pressure, (2) the complexity of maintaining consistency across system calls like fork, and (3) the critical nature of proper cleanup to prevent resource leaks and kernel panics. This implementation demonstrates how high-level features like memory-mapped files are built upon fundamental OS primitives including page tables, interrupts, and file systems.